
KNOWING WHEN TO REVISE: CONFIDENCE-GATED SELF-CORRECTION IN LARGE LANGUAGE MODELS

Fakhr E Alam Khan

University of Trier

Trier, Germany

Matriculation No. 1818211

s2fakhan@uni-trier.de

<https://github.com/Fakharalamkhan/confidence-gated-self-correction>

September 27, 2026

ABSTRACT

Intrinsic self-correction—asking a large language model to critique and revise its own answer without external feedback—often fails to improve accuracy, because revision can turn correct answers into incorrect ones. A natural remedy is confidence gating: revise only when the model’s confidence in its initial answer is low. We give a simple analytical model of gated revision, which shows that the gain of any gate is capped by how often the revision step repairs a wrong answer, and that an uninformative signal can never beat the better of never and always revising. We then compare three training-free confidence signals—verbalized confidence, self-consistency and P(True) self-verification—as gates for Qwen2.5-7B-Instruct on GSM8K and HotpotQA, with thresholds tuned on a development split. Self-critique almost never repairs an error (fix rate 0 on GSM8K, 0.04 on HotpotQA), so no gate improves on never revising. Self-consistency, however, identifies wrong answers well (AUROC 0.95 on GSM8K), and its majority answer repairs 47% of GSM8K errors; used as the revision step, the same gates behave as the analytical model predicts. Our results suggest that the bottleneck of self-correction is the revision step, not the decision of when to revise.

Keywords Large Language Models · Intrinsic Self-Correction · Confidence Estimation · Selective Revision · Test-Time Compute

1 Introduction

A common way to get more out of a large language model (LLM) at inference time is to let it check its own work. The model first produces an answer A_0 , then critiques that answer, and finally writes a revised answer A_1 . Self-Refine [1] and Reflexion [2] made this loop popular, and it is now a standard building block of LLM pipelines. In this paper we study the *intrinsic* version of the loop, in which the model receives no outside help: no code execution, no retrieval, and no ground-truth checker tells it whether A_0 was right.

Whether intrinsic self-correction actually helps is disputed. Huang et al. [3] found that once the oracle signal that earlier work used to decide when to stop is removed, revision on reasoning benchmarks often lowers accuracy. A survey by Kamoi et al. [4] concludes that reliable intrinsic self-correction has so far been shown only in narrow settings, and Stechly et al. [5] report that models are poor judges of their own solutions. The reason becomes clear once each revision is tracked individually. A revision step turns some wrong answers into right ones ($W \rightarrow R$), but it also turns some right answers into wrong ones ($R \rightarrow W$). When initial accuracy is high, there are many more correct answers that can be damaged than wrong answers that can be repaired, so even a small break rate can cancel the benefit. A single net-accuracy number hides this, because a small gain can be the sum of many fixes and many breaks.

This points to a simple remedy: revise only when the model is unsure. We call this *confidence-gated self-correction*. A confidence score $S(A_0)$ is computed for the initial answer, and the revised answer is used if and only if $S(A_0) < \tau$ for a threshold τ ; otherwise A_0 is kept.

The idea itself is not new. Li et al. [6] proposed the If-or-Else (IoE) prompt, which asks the model to keep its answer if it is confident and to revise it otherwise, so that the decision is made inside a single prompt. Yang et al. [7] split self-correction into a *confidence* ability, keeping answers that are already correct, and a *critique* ability, fixing answers that are wrong, and showed that the two can trade off against each other. Two things are still missing. First, there is no clear account of *when* gating can help at all, and how large the gain can be for a given revision step and confidence signal. Second, there is no controlled comparison of different confidence signals used as explicit threshold gates, with thresholds chosen on held-out data and with the cost of computing each signal taken into account.

We address both gaps and organise the study around three research questions:

- **RQ1 (Gating efficacy):** Does confidence gating outperform both never revising and always revising on held-out data, and how close does it come to an oracle gate?
- **RQ2 (Signal discrimination):** Which training-free confidence signal best separates correct from incorrect initial answers?
- **RQ3 (Accuracy–cost trade-off):** How does the cost of computing each signal, split into prefill and completion tokens, change the trade-off between accuracy and compute?

Contributions. This paper makes three contributions.

- **An analytical model of gated revision** (Section 3). We express the accuracy of a gated system in closed form, in terms of the initial accuracy, the fix and break rates of the revision step, and the operating point of the confidence signal on its ROC curve. From this we derive when gating beats never and always revising, show that an uninformative signal can never beat the better of the two, locate the optimal threshold at a fixed slope of the ROC curve, and bound the achievable gain by the fix rate. The model also explains why a high AUROC alone does not guarantee a large gain.
- **A controlled comparison of confidence signals.** We compare three training-free signals—verbalized confidence, self-consistency agreement and P(True) self-verification—on GSM8K and HotpotQA with Qwen2.5-7B-Instruct [8]. Thresholds are tuned on a development split and applied once to a test split, differences are tested with exact McNemar tests, and baselines include IoE, majority voting and an oracle gate.
- **A transition and cost analysis.** We break every revision down into its four possible outcomes, measure cost with a token model that separates prefill from completion tokens, and inspect the $R \rightarrow W$ failures, separating changes in answer form from changes in content.

With Qwen2.5-7B-Instruct, self-critique almost never fixes an error, so no gate beats never revising; self-consistency, in contrast, both identifies wrong answers well and, used as the revision step, repairs many of them, and in that setting gated accuracy matches the analytical model closely.

The rest of the paper is organised as follows. Section 2 reviews related work, Section 3 presents the analytical model, Section 4 describes the experimental setup, Section 5 reports the results, Section 6 analyses them in more depth, and Sections 7 and 8 discuss implications and limitations.

2 Related Work

Self-correction without external feedback. Self-Refine [1] uses one model in three roles—generator, critic and reviser—and iterates between them without any additional training. Reflexion [2] lets an agent write verbal reflections on the feedback it received for a task and keeps them in memory for later attempts. Both reported gains on a range of tasks, but later work questioned how much of the gain comes from the model itself. Huang et al. [3] showed that on GSM8K, CommonSenseQA and HotpotQA, part of the reported improvement came from using the ground-truth label to decide when to stop, and that purely intrinsic self-correction frequently made answers worse. Stechly et al. [5] studied iterative prompting of GPT-4 on graph colouring and found its self-critiques unreliable; most of the improvement came from a correct answer happening to appear among several candidates rather than from the critique itself. Kamoi et al. [4] reviewed the field and concluded that self-correction works reliably when trustworthy external feedback is available or when the model is fine-tuned for it, but that there is little evidence of success with prompting alone. We accept this picture and ask a narrower question: given a revision step that sometimes helps and sometimes hurts, can we decide in advance which answers to revise?

Confidence estimation for LLMs. A gate needs a confidence score, and several training-free ways to obtain one exist. The simplest is to ask the model directly. Xiong et al. [9] evaluated such verbalized confidence together with

sampling-based and hybrid methods across several models, and found that verbalized scores tend to be overconfident, while checking consistency across multiple responses helps to reduce this. Self-consistency [10] samples several reasoning paths and returns the majority answer; the share of samples that agree with a given answer is a natural measure of how sure the model is. Kadavath et al. [11] asked models to judge whether a proposed answer is true and read off the probability of “True”, known as $P(\text{True})$; they found that larger models are well calibrated when questions are posed in a suitable format. These three families—verbalized, sampling-based and probability-based—are the signals we compare. Prior work mostly evaluates them as predictors of correctness; we evaluate them as *decision rules* for whether to revise.

Confidence and self-correction. Two recent papers connect confidence directly to revision. Li et al. [6] identify the model’s confidence as a key hidden factor in intrinsic self-correction and propose the IoE prompt, in which the model assesses its own confidence and decides in the same prompt whether to keep or revise its answer. Yang et al. [7] define probabilistic measures for the confidence and critique abilities, report a trade-off between them under prompting, and propose a fine-tuning data format that improves both. Our study differs from these in three respects. First, the gate is an explicit threshold on a separately computed score rather than a decision left to the revision prompt, so different signals can be swapped in and compared under one protocol. Second, thresholds are selected on a development split and evaluated once on a test split. Third, we give an analytical model that links the ROC curve of a signal to the accuracy gain it can deliver, which explains when and why gating helps.

3 An Analytical Model of Confidence-Gated Revision

Before running any experiment, it helps to ask what a confidence gate can and cannot achieve. This section gives a small model that answers this question with a few quantities that can all be measured on real data.

Setup. Consider one question. Let $Y_0 \in \{0, 1\}$ indicate whether the initial answer A_0 is correct, let $Y_1 \in \{0, 1\}$ indicate whether the revised answer A_1 is correct, and let S be a confidence score computed from A_0 . We describe the revision step by three numbers:

$$\begin{aligned} a &= P(Y_0 = 1) && \text{initial accuracy,} \\ f &= P(Y_1 = 1 \mid Y_0 = 0) && \text{fix rate: how often revision repairs a wrong answer,} \\ b &= P(Y_1 = 0 \mid Y_0 = 1) && \text{break rate: how often revision damages a right answer.} \end{aligned}$$

A gate with threshold τ revises when $S < \tau$. Since the gate should catch wrong answers, we treat “ A_0 is wrong” as the positive class. The gate then has a true-positive rate $t(\tau) = P(S < \tau \mid Y_0 = 0)$, the share of wrong answers it sends to revision, and a false-positive rate $u(\tau) = P(S < \tau \mid Y_0 = 1)$, the share of right answers it sends to revision. As τ varies, the pair (u, t) traces the ROC curve of the signal [12].

Assumption. We assume that, once we know whether A_0 is correct, the outcome of the revision does not depend on the confidence score: $Y_1 \perp S \mid Y_0$. This is a simplification. A model that is unsure about an answer may also be worse at fixing it, which would make the fix rate lower among low-confidence answers. We test how much this matters in Section 6.1.

Gated accuracy. Under this assumption, a correct answer is lost only if the gate sends it to revision and the revision breaks it, and a wrong answer is recovered only if the gate sends it to revision and the revision fixes it. The accuracy of the gated system is therefore

$$\text{Acc}(\tau) = a + (1 - a)ft(\tau) - abu(\tau). \quad (1)$$

The three reference systems are special points of this formula. Never revising is $(u, t) = (0, 0)$ with accuracy a . Always revising is $(u, t) = (1, 1)$ with accuracy $a + (1 - a)f - ab$. The oracle gate, which revises exactly the wrong answers, is $(u, t) = (0, 1)$ with accuracy $a + (1 - a)f$.

When does gating pay off? Define the damage-to-repair ratio

$$\kappa = \frac{ab}{(1 - a)f}, \quad (2)$$

the expected number of correct answers broken for every wrong answer fixed when all answers are revised. Always revising improves on never revising if and only if $\kappa < 1$. Rearranging Eq. (1), a gate operating at (u, t) beats never revising if and only if

$$\frac{t}{u} > \kappa, \quad (3)$$

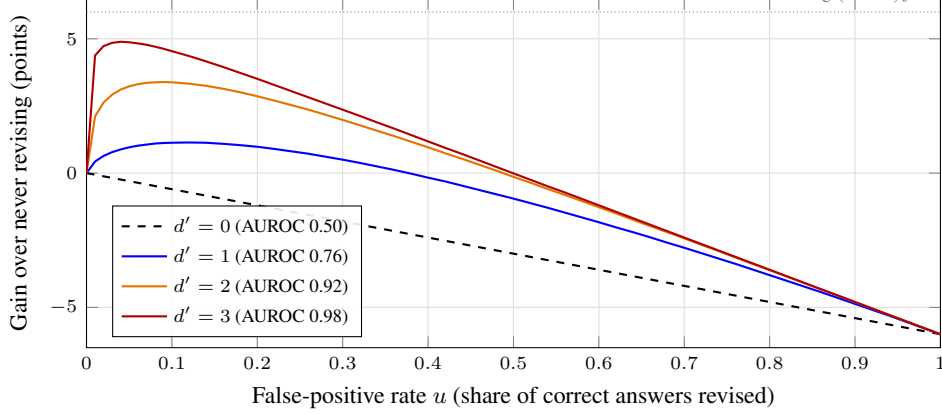


Figure 1: Gain of confidence gating over never revising, as a function of the share of correct answers that are revised, computed from Eq. (1) with hypothetical parameters $a = 0.80$, $b = 0.15$, $f = 0.30$ and binormal ROC curves of separation d' . The right end ($u = 1$) corresponds to always revising (-6 points). The figure illustrates the model only; it does not show experimental data.

and beats always revising if and only if

$$\frac{1-u}{1-t} > \frac{1}{\kappa}. \quad (4)$$

In words: to beat never revising, the gate must send wrong answers to revision at least κ times as often as right ones; to beat always revising, it must hold back right answers at least $1/\kappa$ times as often as wrong ones.

Proposition 1 (No free gating). *If the confidence signal is uninformative, i.e. $t(\tau) = u(\tau)$ for every τ , then for every threshold the gated accuracy lies between the accuracies of never and always revising. An uninformative gate can therefore never beat the better of the two baselines.*

Proof. Setting $t = u = q$ in Eq. (1) gives $\text{Acc} = a + q[(1-a)f - ab]$. This is linear in $q \in [0, 1]$, equals the never-revise accuracy at $q = 0$ and the always-revise accuracy at $q = 1$, and so never leaves the interval between them. $\square$

The optimal threshold. Eq. (1) is linear in (u, t) , so lines of equal accuracy are straight lines of slope κ in ROC space. The best threshold is therefore the point at which the ROC curve has slope κ , the same result as in cost-sensitive ROC analysis [12]. When revision is harmful on average ($\kappa > 1$), this point lies close to the origin: the gate should revise only a small set of answers for which the signal is quite sure that A_0 is wrong.

A ceiling on the gain. Because $t \leq 1$ and $u \geq 0$, the gain of any gate over never revising is at most $(1-a)f$, which is exactly the gain of the oracle gate. If the revision step rarely fixes errors, even a perfect signal can deliver only a small improvement. Put simply, *a gate can remove harm, but it cannot create fixes.*

Why AUROC is not enough. AUROC summarises the whole ROC curve, but by Eq. (1) only the part of the curve near slope κ determines the gain. Two signals with the same AUROC can thus give different gains, and a signal with a high AUROC gives little benefit when the fix rate f is small. For this reason we report both the AUROC of each signal and the accuracy gain it actually achieves.

Illustration. Figure 1 evaluates Eq. (1) for hypothetical values $a = 0.80$, $b = 0.15$ and $f = 0.30$. Here $\kappa = 2$, so always revising costs 6 points, and the oracle ceiling is $(1-a)f = 6$ points. We model the signal with equal-variance binormal ROC curves of separation d' . An uninformative signal ($d' = 0$) only moves along the straight line between the two baselines, as Proposition 1 states. A signal with AUROC 0.76 ($d' = 1$) gains at most about 1.1 points, one with AUROC 0.92 ($d' = 2$) about 3.4 points, and one with AUROC 0.98 ($d' = 3$) about 4.9 points, still below the ceiling. In every case the best operating point revises fewer than 12% of the correct answers.

Table 1: The three confidence signals and what they require per question, in addition to generating A_0 .

Signal	Extra calls	Token probabilities	Main cost	Output
S_{verb}	1	not needed	short completion	integer 0–100
S_{sc}	10	not needed	10 full completions	agreement k/N
S_{verif}	1	needed	prefill (question and context)	$p_A/(p_A + p_B)$

4 Experimental Setup

4.1 Tasks, Data and Scoring

We use two tasks with short answers that can be checked automatically. **GSM8K** [13] consists of grade-school math word problems whose answer is a single number. **HotpotQA** [14] consists of multi-hop questions over Wikipedia. We use its distractor setting, in which each question comes with two gold paragraphs and eight distractor paragraphs; all ten paragraphs are given to the model. We use 500 questions from the GSM8K test set and 500 from the HotpotQA validation set, sampled with seed 42. Each dataset is randomly split (seed 42) into a development split D_{dev} with 20% of the questions, used only to choose thresholds, and a test split D_{test} with the remaining 80%, used once for all reported results.

Every prompt that produces an answer asks the model to finish with a line of the form `Final answer: <answer>`, and only this span is scored. For GSM8K we extract the number, remove formatting such as commas, currency signs and trailing zeros, and compare it with the gold number. For HotpotQA we apply the official answer normalisation (lower-casing and removing punctuation, articles and extra whitespace) and compute exact match (EM) and token-level F1. The correctness label Y is EM on the extracted answer. To measure how much apparent harm is caused only by changes in answer form, we also use a *lenient match*, which counts an answer as correct if the normalised gold answer is contained in the normalised extracted answer.

4.2 Models and Inference

We use the open instruction-tuned model Qwen2.5-7B-Instruct [8] with its own chat template, served with vLLM 0.6.6.post1 on two NVIDIA T4 GPUs (16 GB each) in 16-bit floating point. The initial answer, the revision and the IoE baseline use greedy decoding, so the only randomness in the pipeline comes from the self-consistency samples, each of which is drawn with its own random seed.

4.3 Pipeline and Confidence Signals

For each question we first generate the initial answer A_0 . We then compute three confidence scores $S(A_0) \in [0, 1]$, each in a separate call so that one signal cannot influence another. The full prompts are listed in Appendix A.

1. **Verbalized confidence S_{verb} .** In a new turn after A_0 , the model is asked to state how confident it is that its final answer is correct, as a whole number from 0 to 100, which we divide by 100. If the reply cannot be parsed, we set $S_{\text{verb}} = 1$, so that the answer is never revised by this gate, and we report the number of such cases.
2. **Self-consistency agreement S_{sc} .** We draw $N = 10$ additional samples with the same prompt as A_0 , at temperature 0.7 and top- p 0.95. With k the number of samples whose normalised extracted answer equals that of A_0 , we set $S_{\text{sc}} = k/N$. We also report the variant with $N = 5$, which uses the first five samples.
3. **Self-verification S_{verif} (P(True)).** Following Kadavath et al. [11], the model sees the question (for HotpotQA also the ten paragraphs) together with the extracted answer of A_0 , and must choose between “(A) True” and “(B) False”. With p_A and p_B the next-token probabilities of the two options, summed over their tokenisation variants, we set $S_{\text{verif}} = p_A/(p_A + p_B)$. The gold answer never appears in this prompt.

Finally, the model revises A_0 in the same conversation with a fixed critique-and-revise prompt, producing A_1 . We compute the revision for *every* question, whatever its confidence. This lets us evaluate all gating policies offline on exactly the same outputs, so differences between policies come only from the decision of whether to revise.

Table 1 summarises what each signal needs. The signals differ in two practical ways: whether they require access to token probabilities, and whether their cost comes mainly from reading the input (prefill) or from generating text (completion).

4.4 Gating, Baselines and Threshold Selection

A gate with signal S and threshold τ outputs A_1 if $S(A_0) < \tau$ and A_0 otherwise. We evaluate gates on S_{verb} , S_{sc} and S_{verif} , and on the simple combination $S_{\text{avg}} = \frac{1}{2}(S_{\text{verb}} + S_{\text{sc}})$, which, like its two parts, needs no access to token probabilities. For each gate, the threshold τ^* is chosen on D_{dev} among all observed score values as the one with the highest dev accuracy; ties are broken in favour of the threshold that revises fewer questions. The chosen τ^* is then applied once to D_{test} .

We compare the gates against five baselines:

- **Never revise:** output A_0 .
- **Always revise:** output A_1 .
- **Majority vote:** output the most frequent extracted answer among A_0 and the ten self-consistency samples, a strong baseline that uses the same samples as S_{sc} .
- **IoE** [6]: a single revision prompt that tells the model to keep its answer if it is confident and to change it otherwise, using the prompt of the original paper.
- **Oracle gate:** revise exactly the questions for which A_0 is wrong. It uses the gold labels and serves only as an upper bound, not as a practical method.

Majority vote as a revision step. To test the analytical model with a revision step that does repair errors, we also use the majority answer over the ten self-consistency samples, denoted R_{mv} , in place of A_1 , and gate it with S_{sc} and S_{avg} in exactly the same way. This needs no additional model calls, because the samples are already computed for S_{sc} .

4.5 Metrics and Cost Model

Transitions. Each question falls into one of four cells: $W \rightarrow R$ ($Y_0 = 0, Y_1 = 1$), $R \rightarrow W$ ($Y_0 = 1, Y_1 = 0$), $R \rightarrow R$ or $W \rightarrow W$. For a gated method, only the questions it revises can change cell. From the always-revise transitions we estimate a , f , b and κ as defined in Section 3.

Discrimination. For each signal we report the AUROC for predicting $Y_0 = 1$ on D_{test} , with 95% confidence intervals from 1,000 bootstrap resamples of the test questions.

Significance. Two methods are evaluated on the same questions, so we compare their accuracies with the exact (binomial) McNemar test on the discordant pairs, against both never revising and always revising. We report exact p -values rather than significance stars.

Cost. For question i and a gated method we count

$$C_i = C_i^{A_0} + C_i^S + \mathbb{1}[S_i < \tau] C_i^{\text{rev}}, \quad (5)$$

where $C_i^{A_0}$, C_i^S and C_i^{rev} are the tokens used for the initial answer, for computing the signal and for the revision. We keep prompt (prefill) tokens and generated (completion) tokens separate. Prefill matters here because the verification and revision calls re-read the question and, for HotpotQA, all ten paragraphs. For majority voting the cost is the initial answer plus the ten samples.

5 Results

5.1 Unguided Revision

Table 2: Outcome of unguided (always) revision on the test set ($N = 400$ per dataset). We report initial accuracy $\text{Acc}(A_0)$, transition frequencies ($W \rightarrow R$, $R \rightarrow W$, $R \rightarrow R$, $W \rightarrow W$), accuracy delta $\Delta = \text{Acc}(A_1) - \text{Acc}(A_0)$, empirical fix rate f , break rate b , base accuracy a , and the gating threshold ratio $\kappa = \frac{ab}{(1-a)f}$.

Model	Dataset	N	$\text{Acc}(A_0)$	$W \rightarrow R$	$R \rightarrow W$	$R \rightarrow R$	$W \rightarrow W$	Δ	f	b	κ
Qwen2.5-7B-Instruct	GSM8K	400	92.5%	0 (0.0%)	1 (0.2%)	369 (92.2%)	30 (7.5%)	-0.3%	0.000	0.003	∞
Qwen2.5-7B-Instruct	HotpotQA	400	58.8%	6 (1.5%)	21 (5.2%)	214 (53.5%)	159 (39.8%)	-3.7%	0.036	0.089	3.50

Table 2 shows what happens when every answer is revised. On GSM8K, Qwen2.5-7B-Instruct starts at 92.5% accuracy and revision changes almost nothing: on the 400 test questions it fixes no wrong answer and breaks one correct answer,

so accuracy moves to 92.25%. On HotpotQA revision is clearly harmful: it breaks 21 correct answers and fixes only 6, which lowers accuracy from 58.75% to 55.0% (exact McNemar test against never revising, $p = 0.006$). In the terms of Section 3, GSM8K has a fix rate of $f = 0$, so the oracle ceiling $(1 - a)f$ is zero. HotpotQA has $f = 0.036$ and $b = 0.089$, giving $\kappa = 3.5$: revision damages about 3.5 correct answers for every wrong answer it repairs, and even a perfect gate could gain at most 1.5 points.

5.2 Signal Discrimination (RQ2)

Table 3: AUROC with 95% bootstrap confidence intervals for predicting initial answer correctness $Y_0 = 1$ on the test set ($N = 400$ per dataset). Higher values indicate stronger discrimination.

Model	Dataset	S_{verb}	S_{sc5}	S_{sc10}	S_{verif}	S_{avg}
Qwen2.5-7B-Instruct	GSM8K	0.640 [0.539, 0.762]	0.929 [0.858, 0.993]	0.954 [0.879, 0.994]	0.641 [0.544, 0.730]	0.953 [0.871, 0.995]
Qwen2.5-7B-Instruct	HotpotQA	0.623 [0.564, 0.667]	0.676 [0.633, 0.745]	0.693 [0.647, 0.755]	0.577 [0.521, 0.636]	0.708 [0.655, 0.759]

Table 3 reports how well each signal separates correct from incorrect initial answers. Self-consistency is clearly the strongest signal: with ten samples its AUROC is 0.95 on GSM8K and 0.69 on HotpotQA, and five samples already reach 0.93 and 0.68. Verbalized confidence and P(True) are much weaker, with AUROC between 0.58 and 0.64 on both tasks. Averaging verbalized confidence with self-consistency gives almost the same AUROC as self-consistency alone on GSM8K (0.95) and slightly more on HotpotQA (0.71). On GSM8K, P(True) assigns a low probability to “True” even for most correct answers, so its scores are compressed near zero (Figure 3 in Appendix D).

5.3 Gated Revision (RQ1)

Table 4: Comparison of revision strategies on test splits ($N = 400$ questions each). We report accuracy (Acc), percentage of questions revised (Rev), and exact two-sided McNemar test p -values against the Never-revise baseline (p_{Never}).

Method	GSM8K (Qwen2.5-7B-Instruct)			HotpotQA (Qwen2.5-7B-Instruct)		
	Acc (%)	Rev (%)	p_{Never}	Acc (%)	Rev (%)	p_{Never}
Never	92.5	0.0	—	58.8	0.0	—
Always	92.2	100.0	1.00	55.0	100.0	0.006
Majority vote	95.5	5.5	0.004	60.0	11.5	0.383
IoE	93.0	1.0	0.500	57.2	6.5	0.070
Gate S_{verb}	92.5	0.0	1.00	58.8	0.0	1.00
Gate S_{verb} (hindsight) [†]	92.5	0.0	1.00	58.8	0.0	1.00
Gate S_{sc10}	92.5	0.0	1.00	58.8	4.8	1.00
Gate S_{sc10} (hindsight) [†]	92.5	0.0	1.00	59.5	9.0	0.250
Gate S_{verif}	92.5	0.0	1.00	58.8	0.0	1.00
Gate S_{verif} (hindsight) [†]	92.5	0.0	1.00	58.8	0.0	1.00
Gate S_{avg}	92.5	0.0	1.00	58.8	2.2	1.00
Gate S_{avg} (hindsight) [†]	92.5	0.0	1.00	59.5	10.5	0.250
Oracle	92.5	7.5	1.00	60.2	41.2	0.031

[†]Oracle-tuned upper bound on test set (not a practical method).

Table 4 compares all methods on the test split. When the revision step is self-critique, every gate either selects $\tau^* = 0$ or revises only a few questions, and reproduces the accuracy of never revising; even with the threshold tuned on the test set, no gate exceeds 59.5% on HotpotQA or 92.5% on GSM8K. IoE changes 1.0% of the GSM8K answers (two fixes, no breaks, 93.0%, $p = 0.5$) and 6.5% of the HotpotQA answers, where it fixes one and breaks seven (57.25%, $p = 0.07$). Majority voting is the only method that clearly improves on never revising: it reaches 95.5% on GSM8K ($p = 0.004$) and 60.0% on HotpotQA ($p = 0.38$).

Table 5 uses the majority answer R_{mv} as the revision step instead. This operator has a much higher fix rate: it repairs 14 of the 30 wrong GSM8K answers ($f = 0.47$) and breaks only 3 of the 370 correct ones ($\kappa = 0.21$); on HotpotQA

Table 5: Majority vote as a gated revision operator R_{mv} on GSM8K and HotpotQA (Qwen2.5-7B-Instruct, test split). τ^* tuned on development split. p_{Never} is two-sided exact McNemar test vs. never revising. Predicted vs. observed test accuracy from the analytical model.

Dataset	Gate	τ^*	Acc (%)	Rev (%)	$W \rightarrow R$	$R \rightarrow W$	p_{Never}	Pred (%)	Obs (%)
GSM8K	Gate S_{sc}	0.000	92.5	0.0	0	0	1.00	92.50	92.50
GSM8K	Gate S_{avg}	0.000	92.5	0.0	0	0	1.00	92.50	92.50
HotpotQA	Gate S_{sc}	0.200	59.5	9.0	7	4	0.55	59.56	59.50
HotpotQA	Gate S_{avg}	0.500	59.5	8.8	6	3	0.51	59.58	59.50

it fixes 18 and breaks 14 ($\kappa = 0.78$). On HotpotQA, the gate on S_{sc} revises only 9.0% of the questions and reaches 59.5% (7 fixes, 4 breaks), close to ungated majority voting while revising far fewer questions; the difference from never revising is not significant ($p = 0.55$). On GSM8K, no threshold improved accuracy on the 100 development questions, so the development-tuned gate does not revise at all; with the threshold tuned on the test set, the same gate would reach 96.0%, above ungated majority voting.

5.4 Accuracy–Cost Trade-off (RQ3)

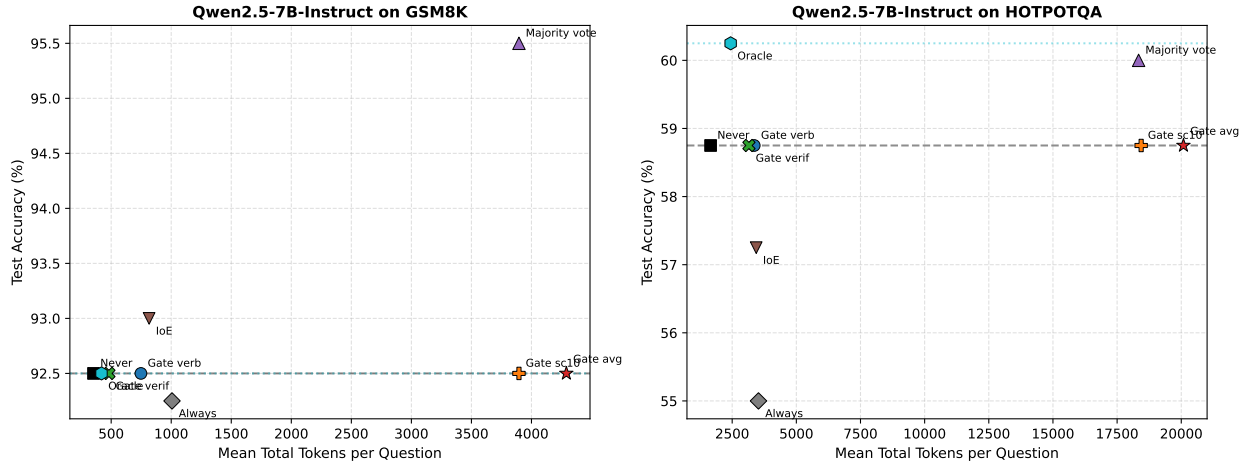


Figure 2: Accuracy versus compute cost (mean total tokens per question, prefill plus completion) across baseline and confidence-gated revision policies for Qwen2.5-7B-Instruct on GSM8K and HotpotQA. Dashed lines indicate the Never-revise baseline and the Oracle ceiling.

Figure 2 relates accuracy to the mean number of tokens per question. The methods fall into two cost groups. Never revising costs about 350 tokens per GSM8K question and about 1,660 per HotpotQA question; self-critique, IoE, verbalized confidence and $P(\text{True})$ add roughly one more pass over the conversation. Everything based on self-consistency—the S_{sc} gates and majority voting—needs about eleven times the tokens of a single answer, because ten full answers are sampled. $P(\text{True})$ generates only a single token, but on HotpotQA it re-reads all ten paragraphs and roughly doubles the prompt tokens (about 3,000 instead of 1,500). The accuracy gains of majority voting therefore come at a substantial cost.

6 Analysis

6.1 Does the Analytical Model Predict Gated Accuracy?

With self-critique as the revision step, the development-tuned gates revise almost no questions, so their operating points (u, t) lie close to the origin and Eq. (1) reproduces the observed accuracies trivially (Table 9 in Appendix D). The ceiling is the informative prediction here: no gate can improve on never revising by more than $(1 - a)f$, which is 0 points on GSM8K and 1.5 points on HotpotQA, and no gate does. Majority voting as the revision step provides a non-trivial test, because its gates revise about 9% of the HotpotQA questions. Inserting the measured a , f_{mv} , b_{mv} and

the gate’s (u, t) into Eq. (1) predicts 59.56% for the S_{sc} gate and 59.58% for the S_{avg} gate; the observed accuracy is 59.50% for both (Table 5). The independence assumption of Section 3 thus appears adequate at this scale.

6.2 Anatomy of Harmful Revisions

Table 6: Analysis of harmful revisions ($R \rightarrow W$) under always-revise on HotpotQA ($N = 400$). Comparing exact match (EM) against lenient string containment reveals how many apparent errors arise from surface format drift rather than semantic loss.

Model	Always-Revise $R \rightarrow W$ (EM)	Always-Revise $R \rightarrow W$ (Lenient)	Format Drift Cases
Qwen2.5-7B-Instruct	21	17	4

The main reason why gating cannot help with self-critique is that the revision step rarely changes anything. On GSM8K, the revised answer differs from the initial one for only 2 of 500 questions, and for only 1 of the 38 questions whose initial answer is wrong. Reading these cases shows that the model re-checks its solution and confirms it, even when it is wrong. For question `gsm8k-test-255` (gold answer 192), the initial solution computes $8 \times 12 = 96$ cows; the revision repeats the same step and ends with “Upon review, the steps are correct.” On HotpotQA, revision changes 11.4% of the answers, but these changes break more answers than they fix. Under lenient matching, 4 of the 21 HotpotQA $R \rightarrow W$ cases are no longer counted as errors, so part of the apparent harm is a change in answer form rather than in content. Score histograms and threshold-sensitivity curves are given in Appendix D.

7 Discussion

The bottleneck is the revision, not the gate. The analytical model separates two questions that are easy to confuse: whether revising helps on the questions it touches, measured by the fix and break rates, and whether a confidence signal can find those questions, measured by its ROC curve. Our results show that the first question decides the outcome. With self-critique, the fix rate is zero on GSM8K and very small on HotpotQA, so no signal can make revision useful, even though self-consistency identifies wrong answers well. With majority voting as the revision step, the same signals and the same gates produce gains that match the model’s prediction. A gate can remove harm, but it cannot create fixes.

Why self-critique fails here. The revised answers show a clear pattern of self-confirmation: the model repeats its earlier reasoning and declares it correct. Part of this is probably caused by our revision prompt, which explicitly allows the model to keep its answer when it finds no error. A more critical prompt might raise the fix rate, but, as the HotpotQA results suggest, it would probably raise the break rate as well. Measuring both rates is more informative than reporting net accuracy alone.

Answers to the research questions. **RQ1:** With self-critique as the revision step, no gate outperforms never revising; with majority voting as the revision step, gating keeps most of the benefit of majority voting while revising far fewer questions, but its advantage over never revising is not significant on our test sets, and a small development split can fail to find a useful threshold. **RQ2:** Self-consistency is the most discriminative signal (AUROC 0.95 on GSM8K and 0.69 on HotpotQA); verbalized confidence and $P(\text{True})$ are weak (0.58–0.64). **RQ3:** The best signal is also the most expensive: self-consistency needs about eleven times the tokens of a single answer, whereas $P(\text{True})$ is cheap to generate but expensive to prefill when the context is long.

Comparison with prior work. Our always-revise results agree with Huang et al. [3]: without external feedback, revision does not improve reasoning accuracy and can reduce it. In the terms of Yang et al. [7], Qwen2.5-7B-Instruct shows high confidence (it rarely changes a correct answer) but very low critique ability (it almost never repairs a wrong one). IoE [6] behaves like a cautious revision step: it changes few answers and, on HotpotQA, still breaks more than it fixes. The strength of majority voting is consistent with Wang et al. [10].

Practical recommendation. Before adding a confidence gate to a self-correction pipeline, practitioners should estimate the fix and break rates of the revision step on a development set. If $(1 - a)f$ is small, the expected benefit of any gate is small as well, and effort is better spent on a revision step that actually repairs errors, such as resampling, or on obtaining external feedback.

8 Limitations

Our study has several limitations that bound how far its conclusions reach.

- **Models.** We study a single open instruction-tuned model with 7B parameters, so our conclusions may not transfer to other model families or sizes. Larger models are reported to judge their own answers more reliably [11] and may have different fix and break rates, so the ranking of signals could change with scale. The model was run in 16-bit floating point on T4 GPUs; small numerical differences from other hardware or precisions are possible.
- **Tasks.** Both tasks are in English and have short answers that can be checked automatically. Open-ended generation, where correctness needs a semantic judgement, is outside our scope.
- **Sample size.** Each dataset contributes a few hundred test questions. This is enough to compare methods with paired tests, but confidence intervals are wide, and small accuracy differences between gates should not be over-interpreted.
- **Prompts.** We use one critique-and-revise prompt and a single round of revision. Fix and break rates depend on the wording of the revision prompt [7], so other prompts or multi-round revision could shift the results.
- **Labels.** Exact match can mark a correct but differently phrased answer as wrong. This adds noise to the correctness labels, to the transition counts and to AUROC. We quantify the effect with a lenient match on HotpotQA, but do not remove it.
- **Analytical model.** The model in Section 3 assumes that the outcome of revision is independent of the confidence score once correctness is known. Section 6.1 tests how well this holds; where it fails, the model’s predictions are only approximate.
- **Threshold selection.** The development split is small compared with the number of candidate thresholds, so the chosen threshold τ^* has sampling variance and may not be optimal on the test split.
- **Cost.** Token counts are a proxy for cost. Wall-clock time depends on hardware, batching and prefix caching, which can make repeated prefill much cheaper than our token counts suggest.

9 Conclusion

We asked when a language model should revise its own answer. Our analytical model shows that confidence gating can only help when the revision step actually repairs errors and the confidence signal is informative in the relevant region of its ROC curve; an uninformative signal can never beat the better of never and always revising, and the achievable gain is capped by the fix rate. Experiments with Qwen2.5-7B-Instruct confirm both sides of this picture. Self-critique almost never repairs an error, so no gate improves on never revising, even though self-consistency identifies wrong answers well. When the majority answer of the self-consistency samples is used as the revision step instead, the same gates behave as the model predicts. Future work should therefore focus on the revision step itself, and on selecting thresholds reliably from small development sets.

References

- [1] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- [2] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- [3] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. In *International Conference on Learning Representations (ICLR)*, 2024.
- [4] Ryo Kamoi, Yusen Zhang, Nan Zhang, Jiawei Han, and Rui Zhang. When can LLMs actually correct their own mistakes? A critical survey of self-correction of LLMs. *Transactions of the Association for Computational Linguistics*, 12:1417–1440, 2024.

- [5] Kaya Stechly, Matthew Marquez, and Subbarao Kambhampati. GPT-4 doesn’t know it’s wrong: An analysis of iterative prompting for reasoning problems. *arXiv preprint arXiv:2310.12397*, 2023.
- [6] Loka Li, Zhenhao Chen, Guangyi Chen, Yixuan Zhang, Yusheng Su, Eric Xing, and Kun Zhang. Confidence matters: Revisiting intrinsic self-correction capabilities of large language models. *arXiv preprint arXiv:2402.12563*, 2024.
- [7] Zhe Yang, Yichang Zhang, Yudong Wang, Ziyao Xu, Junyang Lin, and Zhifang Sui. Confidence v.s. critique: A decomposition of self-correction capability for LLMs. *arXiv preprint arXiv:2412.19513*, 2024.
- [8] Qwen Team, An Yang, Baosong Yang, Beichen Zhang, et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.
- [9] Miao Xiong, Zhiyuan Hu, Xinyang Lu, Yifei Li, Jie Fu, Junxian He, and Bryan Hooi. Can LLMs express their uncertainty? An empirical evaluation of confidence elicitation in LLMs. In *International Conference on Learning Representations (ICLR)*, 2024.
- [10] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [11] Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, et al. Language models (mostly) know what they know. *arXiv preprint arXiv:2207.05221*, 2022.
- [12] Tom Fawcett. An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8):861–874, 2006.
- [13] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- [14] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 2369–2380, 2018.

A Prompts

All prompts are given verbatim; {...} marks a placeholder filled per question. Each prompt is sent as a user turn with the model’s own chat template.

A0 GSM8K

```
Solve the following problem step by step. At the end, write your answer on a new line as:
Final answer: <answer>

Problem: {question}
```

A0 HotpotQA

```
Answer the question using the paragraphs below. Think step by step. At the end, write your
answer on a new line as:
Final answer: <answer>
The answer should be a short span (a name, date, number, or yes/no).

Paragraphs:
{context}

Question: {question}
```

Verbalized confidence

```
How confident are you that your final answer above is correct? Reply with a single integer
from 0 to 100 and nothing else.
```

Self-verification P(True)

```
Question: {question}
Proposed answer: {extracted_A0_answer}
Is the proposed answer correct?
(A) True
```

(B) False
Answer with the letter only.

The assistant turn is prefilled with “The proposed answer is: (” and the next-token probabilities of “A” and “B” are read.

Revision

Review your previous answer carefully. Check each step for errors. If you find a specific error, correct it. If you find no error, keep your original answer. End with a new line: Final answer: <answer>

IoE

Review your previous answer. If you are very confident about your answer, maintain your answer. Otherwise, update your answer. End with a new line: Final answer: <answer>

Adapted from Li et al. [6].

B Implementation Details

All experiments were run on Kaggle with two NVIDIA T4 GPUs (16 GB each), using vLLM 0.6.6.post1 with tensor parallelism over both GPUs, PyTorch 2.5.1+cu121, Transformers 4.48.3, 16-bit floating point and a maximum context length of 4,096 tokens. Each of the ten self-consistency samples uses its own random seed (42–51). Before every full run, a dry run on 20 questions checked that the gold answer never appears in the verification prompt, that all stages are aligned by question identifier, that a random score yields an AUROC near 0.5, and that the self-consistency samples are diverse. No verbalized-confidence or P(True) output failed to parse on the dry runs. The code, prompts, data splits and raw outputs are available at <https://github.com/Fakharalamkhan/confidence-gated-self-correction>; the README lists the exact commands to reproduce all tables and figures.

C Contribution Summary and Use of AI Tools

Contribution summary. This is individual work. The author formulated the research question, designed the study (the three confidence signals, the tasks, the four-way transition analysis, the dev/test protocol and the baselines), ran and monitored all experiments, checked the results, and is responsible for all claims in the paper.

Use of AI tools. AI assistants were used for coding support in implementing the experiment pipeline, for help with drafting, restructuring and editing the text, and for checking bibliographic details. All AI-generated code and text were reviewed and revised by the author. No AI tool was used to produce data, labels or reported numbers; all results come from the logged experiment outputs in the repository (<https://github.com/Fakharalamkhan/confidence-gated-self-correction>).

D Complete Numerical Results across All Experimental Settings

This appendix provides the complete tabular breakdown of accuracy, revision rates, transition frequencies ($W \rightarrow R$ and $R \rightarrow W$), token costs, and exact McNemar significance tests across all evaluated methods and confidence gates for Qwen2.5-7B-Instruct on GSM8K and HotpotQA, as well as the analytical model verification, score distributions, and threshold sensitivity curves.

D.1 Full Gating Results on GSM8K

Table 7: Full gating results for Qwen2.5-7B-Instruct on GSM8K ($N = 400$ test questions).

Method	τ^*	Acc (%)	Rev (%)	$W \rightarrow R$	$R \rightarrow W$	Prompt Tok	Comp Tok	p_{Never}	p_{Always}
Never	–	92.5	0.0	0	0	121	232	–	1.00
Always	–	92.2	100.0	0	1	528	479	1.00	–
Majority vote	–	95.5	5.5	14	2	1326	2570	0.004	0.002
IoE	–	93.0	1.0	2	0	519	297	0.500	0.250
Gate S_{verb}	0.000	92.5	0.0	0	0	512	236	1.00	1.00
Gate S_{verb} (hindsight) [†]	0.000	92.5	0.0	0	0	512	236	1.00	1.00
Gate S_{sc10}	0.000	92.5	0.0	0	0	1326	2570	1.00	1.00
Gate S_{sc10} (hindsight) [†]	0.000	92.5	0.0	0	0	1326	2570	1.00	1.00
Gate S_{verif}	0.000	92.5	0.0	0	0	248	233	1.00	1.00
Gate S_{verif} (hindsight) [†]	0.000	92.5	0.0	0	0	248	233	1.00	1.00
Gate S_{avg}	0.000	92.5	0.0	0	0	1717	2574	1.00	1.00
Gate S_{avg} (hindsight) [†]	0.000	92.5	0.0	0	0	1717	2574	1.00	1.00
Oracle	–	92.5	7.5	0	0	160	259	1.00	1.00

[†]Oracle-tuned upper bound on test set (not a practical method).

D.2 Full Gating Results on HotpotQA

Table 8: Full gating results for Qwen2.5-7B-Instruct on HotpotQA ($N = 400$ test questions).

Method	τ^*	Acc (%)	Rev (%)	$W \rightarrow R$	$R \rightarrow W$	Prompt Tok	Comp Tok	p_{Never}	p_{Always}
Never	–	58.8	0.0	0	0	1510	149	–	0.006
Always	–	55.0	100.0	6	21	3223	301	0.006	–
Majority vote	–	60.0	11.5	13	8	16611	1726	0.383	0.002
IoE	–	57.2	6.5	1	7	3214	222	0.070	0.108
Gate S_{verb}	0.000	58.8	0.0	0	0	3207	152	1.00	0.006
Gate S_{verb} (hindsight) [†]	0.000	58.8	0.0	0	0	3207	152	1.00	0.006
Gate S_{sc10}	0.100	58.8	4.8	0	0	16701	1736	1.00	0.006
Gate S_{sc10} (hindsight) [†]	0.200	59.5	9.0	3	0	16701	1736	0.250	2.8e-04
Gate S_{verif}	0.000	58.8	0.0	0	0	3004	150	1.00	0.006
Gate S_{verif} (hindsight) [†]	0.000	58.8	0.0	0	0	3004	150	1.00	0.006
Gate S_{avg}	0.425	58.8	2.2	0	0	18351	1734	1.00	0.006
Gate S_{avg} (hindsight) [†]	0.575	59.5	10.5	3	0	18351	1734	0.250	2.8e-04
Oracle	–	60.2	41.2	6	0	2227	214	0.031	9.5e-07

[†]Oracle-tuned upper bound on test set (not a practical method).

D.3 Analytical Model Verification

Table 9: Comparison of predicted gated accuracy from the analytical model (Eq. 1) against empirically observed gated accuracy on the test set ($N = 400$ per dataset).

Model	Dataset	Gate	τ^*	TPR (t)	FPR (u)	Pred Acc (%)	Obs Acc (%)	Diff (%)
Qwen2.5-7B-Instruct	GSM8K	S_{verb}	0.000	0.000	0.000	92.50	92.50	+0.00
Qwen2.5-7B-Instruct	GSM8K	S_{sc10}	0.000	0.000	0.000	92.50	92.50	+0.00
Qwen2.5-7B-Instruct	GSM8K	S_{verif}	0.000	0.000	0.000	92.50	92.50	+0.00
Qwen2.5-7B-Instruct	GSM8K	S_{avg}	0.000	0.000	0.000	92.50	92.50	+0.00
Qwen2.5-7B-Instruct	HotpotQA	S_{verb}	0.000	0.000	0.000	58.75	58.75	+0.00
Qwen2.5-7B-Instruct	HotpotQA	S_{sc10}	0.100	0.103	0.009	58.86	58.75	-0.11
Qwen2.5-7B-Instruct	HotpotQA	S_{verif}	0.000	0.000	0.000	58.75	58.75	+0.00
Qwen2.5-7B-Instruct	HotpotQA	S_{avg}	0.425	0.055	0.000	58.83	58.75	-0.08

D.4 Score Distributions and Threshold Sensitivity

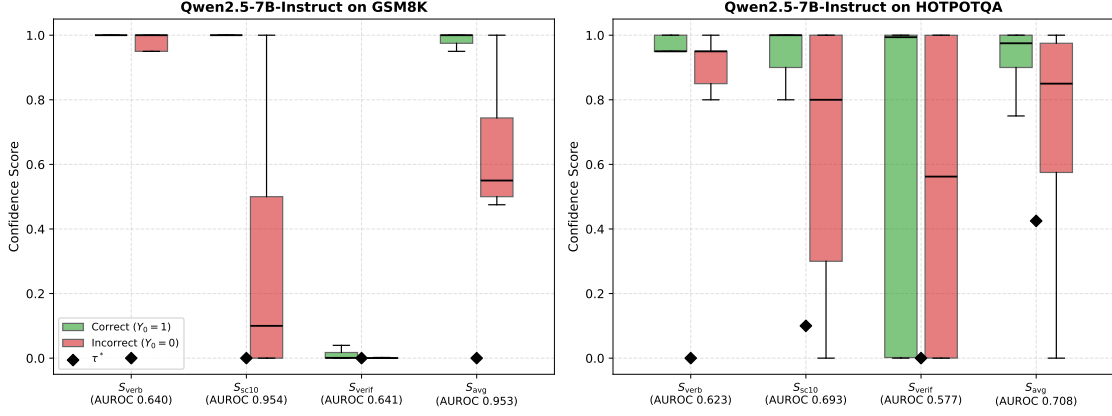


Figure 3: Distributions of confidence scores for correct ($Y_0 = 1$) versus incorrect ($Y_0 = 0$) initial answers across signals on GSM8K (left) and HotpotQA (right) for Qwen2.5-7B-Instruct. Diamonds indicate the dev-selected threshold τ^* .

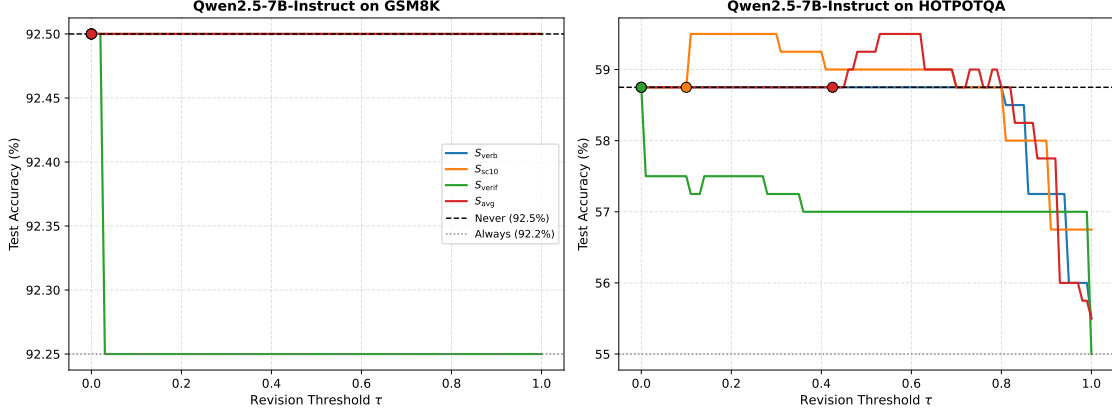


Figure 4: Sensitivity of test accuracy to revision threshold τ for each confidence signal on GSM8K (left) and HotpotQA (right). Markers indicate operating points at the dev-selected threshold τ^* .